

How I connected dot to iMessage with Photon

I connected dot to iMessage using Photon. The starting point was my existing photongrokbob repo, which already handled message routing, replies, reactions, polls, image cards and voice notes.

The adaptation keeps that runtime and gives dot a clear way to receive work and send a response back. Photon handles the iMessage connection. The bridge keeps track of messages and delivery attempts. An active dot task reads the incoming work, decides what to do and authors the reply.

Here's the path through the system.

```
iMessage
  → Photon Spectrum
  → existing Bun bridge
  → saved batch and local inbox notice
  → active dot task claims and processes the batch
  → authorized actions enter the existing outbound queue
  → Photon Spectrum
  → the original iMessage conversation
```

That “active dot task” step matters. This build uses a local inbox that dot can inspect while a task is running. It has no automatic platform wake endpoint. If the bridge stays up while dot is inactive, batches can wait in the inbox. If the bridge itself is stopped, this implementation does not guarantee replay of messages sent during that gap.

Keeping the existing message pipeline

The repo already had the pieces that make an iMessage assistant useful beyond a plain text exchange. It knows how to handle attachments, transcribe audio, group image cards and associate a reaction with the option it belongs to.

Those modules stayed in place. The main change is the connection between the bridge and the task doing the reasoning.

In the original deployment, a flushed batch could notify the Grok runtime. In dot mode, it writes a durable local notice containing the batch ID. The full payload stays in the batch store. dot reads it, takes ownership and produces actions through the existing queue.

The bridge uses the official Photon Spectrum SDK, with both @spectrum-ts/core and @spectrum-ts/imessage pinned to version 12.10.1. Both directions use the same runtime and provider connection. The new interface between dot and the bridge is small enough to inspect without replacing the rest of the message pipeline.

Saving work before processing it

An incoming event passes through sender, direction, platform and conversation checks. The runtime then persists the message before recording that it has been handled.

That ordering matters if the process stops between two writes. Marking a message handled before its contents are saved could make it disappear from the next run. The saved inbound records give recovery something concrete to work from.

Messages are batched before dot sees them. The bridge writes the full batch and a small inbox notice. The active task can wait on that inbox directly. The wait command returns one batch with an exclusive claim, or times out.

The claim has an owner and an expiry time, and the same owner can renew it while work continues. That gives one task responsibility for a batch even if more than one local process is looking for work.

The conversation owner can ask a specialist to prepare an image set, build an app sheet or investigate a code issue. The result comes back to that owner, which checks the proposed response and enqueues it. This keeps two workers from independently replying to the same request.

The original repo described six agent roles. In this adaptation, those become responsibilities assigned as needed within the active task. The bridge does not start six permanent bots.

Making retries predictable

The reply command checks live claim ownership and the original destination. For text, it accepts standard input, durably enqueues the response and completes the batch.

Each action also gets a stable key formed from the batch ID and an action ID. Retrying the same action with the same input returns the existing queue record. Reusing the key with different input is rejected.

This is useful when a caller loses track of whether its enqueue succeeded. It can ask again with the same key without creating another copy of the response.

Delivery has a separate set of states. A queued action has been saved locally. A provider-accepted action has reached Spectrum. A correlated read event gives additional evidence that the recipient read the message.

These states stay separate in the records. Completing a batch means the task finished processing it; it does not mean the recipient has read its answer.

There is also an unknown state. Imagine a send reaches the provider, but the process loses the result before it can save the returned message ID. Automatically sending again could deliver the same answer twice. The bridge holds an uncertain attempt for reconciliation instead of replaying it.

The same rule applies to threaded replies. If a reply throws after an uncertain send, immediately falling back to a plain message could duplicate it. Fallback is restricted to cases where the first operation is known not to have sent.

This is a filesystem-backed recovery design. Atomic JSON replacements, flushed audit records and local process locks make the state inspectable and help recover interrupted work. They do not establish exactly-once delivery across a network.

Voice notes enter the same conversation

Voice uses the existing local Moonshine pipeline. For incoming CAF or m4a audio, the bridge uses ffmpeg to create a 16 kHz mono WAV and runs Moonshine locally. The transcript becomes part of the inbound work that dot reads.

The prepared environment uses moonshine-voice 0.1.5 and the quantized small English streaming model. All nine required model files were downloaded from the official asset repository and checked against the publisher's hashes.

For a real local test, a 44.37-second spoken fixture went through the CAF-to-WAV-to-transcript path in about 8.8 seconds. It produced a coherent transcript with some recognition and punctuation errors.

That verifies the local audio pipeline on this machine. A voice note sent from a phone through the full running bridge still needs its own end-to-end check.

Preserving richer replies

The outbound queue retains the existing response types: text and threaded replies, reactions, effects, polls, voice, grouped attachments and app surfaces.

A poll can handle a bounded choice. An image-card set can make several options easier to compare. Card metadata stays aligned with attachment order so a reaction maps back to the right option and its saved details.

App sheets provide a larger surface when the task calls for one. The optional Live Mini package can display progress at a stable URL, with updates to the card's saved state rather than a new chat message for every milestone.

Live Mini still needs an authorized HTTPS host and durable private storage before a phone can use it. Local builds and HTTP checks establish that the package works locally; they do not establish device rendering or a live connection to task progress.

What has been verified

The adapted runtime is running and connected to hosted iMessage through Spectrum. The earlier minimal adapter was stopped before the full runtime took over, leaving one provider connection in this setup.

The bridge passes its automated and legacy assertion suites, TypeScript checks and local prerequisite checks. Preflight verifies all nine Moonshine model files against their recorded SHA-256 hashes. The tests cover claim ownership, duplicate enqueue attempts, changed-payload rejection, interrupted work, ambiguous sends, receipt correlation and the new timing and direct-reply paths.

Live Mini separately passes its application and skill tests, build and local HTTP checks. It has not been publicly deployed or checked on a phone.

The full runtime has a verified dot-authored text round trip through a claimed batch and the existing outbox, followed by Spectrum acceptance and a matching read receipt. The automatic greeting also completed its return path. The first exchange had a user-observed wait of about 93 seconds. The conversation owner now waits on the inbox and replies directly. New timestamps separate stream entry, context loading, durable saving and SDK send return. Acceptance is persisted before bounded, asynchronous typing cleanup. These changes are loaded, but still need a fresh speed measurement. Phone voice notes and rich rendering remain unverified.

The remaining device checks matter because a passing test cannot show how Messages renders a poll, grouped images, a voice note or an app sheet. Each needs to pass through the actual provider and arrive on a phone.

The result is a bridge with a clear division of work. Photon carries the messages. The runtime records what arrived and what it tried to send. dot owns the response. The existing iMessage conversation is where the user sees the outcome.